PAPER_377 — compute_a_wormhole() Implementation & MUGE Safety Infrastructure

Source: grok_share_11254865.txt, lines 8600–10322 (C++ v8 and v9 final programs) **Session:** 102 (final unread block, confirmed from complete file read) **CP4 Class:** `WormholeMUGETermImplSafetyCalculator` (CP4 #26)

Abstract

$$F_{U,Bi} = \kappa \cdot \frac{\rho_{\text{SCm}}}{\rho_{\text{UA}}} \cdot (U_{g1} + U_{g2} + U_{g3} + U_{g4} + U_m + U_{bi})$$

This paper presents a UQFF analysis of compute_a_wormhole() Implementation & MUGE Safety Infrastructure, deriving compressed field equations and observational predictions within the Star-Magic/UQFF framework.

1. Overview

This paper captures the formal C++ implementation of the wormhole coupling term in the Resonance MUGE framework, along with division-by-zero error safety in the Compressed MUGE functions, a complete 24-test assertion suite, and full CSV I/O for MUGESystem data. These form the production-ready computational layer for the UQFF framework.

---

2. compute_a_wormhole() — Implemented Function

Canonical Form

```
double compute_a_wormhole(double r, double b = 1.0,
                          double f_worm = 1.0,
                          double Evac_neb = 7.09e-36) {
    return f_worm * Evac_neb * (1.0 / (b * b + r * r));
}
```

Physical Meaning

`a_worm = f_worm · Evac_neb / (b² + r²)`

- `b = 1.0 m` — Morris-Thorne throat radius (PAPER_373 baseline)
- `f_worm = 1.0` — wormhole coupling constant (dimensionless, unit default)
- `Evac_neb = 7.09×10⁻³⁶ J/m³` — nebular vacuum energy density
- Denominator `(b² + r²)` — wormhole geometry: r→0 gives throat maximum, r→∞ → 0

Numerical values:

```
At r = 1e4 m,  b = 1.0:  a_worm = 7.09e-36 / (1 + 1e8) ≈ 7.09e-44 m/s²
At r = 1e12 m, b = 1.0:  a_worm = 7.09e-36 / 1e24 = 7.09e-60 m/s²
At r = 0.0 m,  b = 1.0:  a_worm = 7.09e-36 / 1.0  = 7.09e-36 m/s²  (throat max)
```

Integration into compute_resonance_MUGE()

```
double compute_resonance_MUGE(const MUGESystem& sys, const ResonanceParams& res) {
    double aDPM          = compute_aDPM(sys, res);
    double aTHz          = compute_aTHz(aDPM, sys, res);
    double avac_diff     = compute_avac_diff(aDPM, sys, res);
    double asuper_freq   = compute_asuper_freq(aDPM, res);
    double aaether_res   = compute_aaether_res(aDPM, res);
    double Ug4i          = compute_Ug4i(aDPM, sys, res);
    double aquantum_freq = compute_aquantum_freq(aDPM, res);
    double aAether_freq  = compute_aAether_freq(aDPM, res);
    double afluid_freq   = compute_afluid_freq(sys, res);
    double Osc_term      = compute_Osc_term();
    double aexp_freq     = compute_aexp_freq(aDPM, sys, res);
    double fTRZ          = compute_fTRZ(res);
    double a_worm        = compute_a_wormhole(sys.r);   // ← NEW final term

    return aDPM + aTHz + avac_diff + asuper_freq + aaether_res
         + Ug4i + aquantum_freq + aAether_freq + afluid_freq
         + Osc_term + aexp_freq + fTRZ + a_worm;  // 13 total terms
}
```

---

3. Error-Safe Compressed MUGE Functions

Division-by-zero protection added to 4 compressed MUGE functions:

```
double compute_compressed_base(const MUGESystem& sys) {
    if (sys.r == 0.0)
        throw std::runtime_error("Division by zero in r");
    return G * sys.M / (sys.r * sys.r);
}

double compute_compressed_super_adj(const MUGESystem& sys) {
    if (sys.Bcrit == 0.0)
        throw std::runtime_error("Division by zero in Bcrit");
    return 1 - sys.B / sys.Bcrit;
}

double compute_compressed_quantum(double hbar, double Delta_x_p, ...) {
    if (Delta_x_p == 0.0)
        throw std::runtime_error("Division by zero in Delta_x_p");
    return (hbar / Delta_x_p) * integral_psi * (2 * PI / tHubble);
}

double compute_compressed_perturbation(const MUGESystem& sys) {
    if (sys.r == 0.0)
        throw std::runtime_error("Division by zero in r^3");
    return (sys.M + sys.M_DM) * (sys.delta_rho_rho + 3*G*sys.M / (sys.r*sys.r*sys.r));
}
```

---

4. Complete 24-Test Assertion Suite

The final test suite (`run_unit_tests()`) contains 24 tests:

Test Function	Expected Value	Source
test_compute_compressed_base	$G \times M_{sun}/(1AU)^2 \approx 0.0059$	Newtonian validation
test_compute_compressed_exp_const	1.0 (at t=0)	Zero-time boundary
test_compute_compressed_super_freq	0.9 (B=1e10, Bcrit=1e11)	B/Bcrit = 0.1
test_compute_compressed_fluid	4.189e-2	$\rho \times V \times g$ product
test_compute_compressed_env	1.0	Identity
test_compute_compressed_Ug_osc	0.0	Simplified
test_compute_compressed_cosm	$1.1e-52 \times c^2/3$	Λ constant
test_compute_compressed_quantum	$(1/1e68) \times 2.176e-18 \times (2\pi/4.35e17)$	Hubble time
test_compute_compressed_perturbation	$M \times (1 + 3GM/r^3)$	SGR1745 params
test_compute_compressed_MUGE	1.782e39 m/s^2	SGR1745 vs document
test_compute_aDPM	3.545e-42 m/s^2	SGR1745
test_compute_aTHz	1.182e-33 m/s^2	aDPM=3.545e-42, vexp=1e3
test_compute_avac_diff	3.545e-53 m/s^2	aDPM=3.545e-42, vexp=1e3
test_compute_asuper_freq	1.048e-21 m/s^2	aDPM=3.545e-42
test_compute_aaether_res	3.900e-38 m/s^2	aDPM=3.545e-42
test_compute_Ug4i	0.0 (Ereact≈0 at t=3.799e10)	Decay asymptote
test_compute_aquantum_freq	1.708e-66 m/s^2	Hubble resonance scale
test_compute_aAether_freq	1.863e-84 m/s^2	Aether freq
test_compute_afluid_freq	1.773e-9 m/s^2	SGR1745 afluid confirmation
test_compute_Osc_term	0.0	Identity
test_compute_aexp_freq	1.623e-57 m/s^2	SGR1745 at t=3.799e10
test_compute_fTRZ	0.1	Default value
test_compute_resonance_MUGE	1.773e-9 m/s^2	SGR1745 total resonance
test_compute_a_wormhole	**1/(1+r^2) (at Evac_neb=1, b=1)**	**NEW in v9**

```
void test_compute_a_wormhole() {
    double r = 1e4;
    double b = 1.0;
    double expected = 1.0 / (1.0 + r * r);  // f_worm=1, Evac_neb=1 for test
    double result = compute_a_wormhole(r, b, 1.0, 1.0);
    assert(std::abs(result - expected) < 1e-6);
}
```

---

5. CSV File I/O — load_muge_systems()

Complete 18-field CSV parser for MUGESystem:

```
std::vector<MUGESystem> load_muge_systems(const std::string& filename) {
    std::vector<MUGESystem> systems;
    std::ifstream in(filename);
    if (in.is_open()) {
```

```
        std::string line;
        while (std::getline(in, line)) {
            std::stringstream ss(line);
            MUGESystem sys;
            std::string token;
            std::getline(ss, sys.name, ',');
            std::getline(ss, token, ','); sys.I            = std::stod(token);
            std::getline(ss, token, ','); sys.A            = std::stod(token);
            std::getline(ss, token, ','); sys.omega1       = std::stod(token);
            std::getline(ss, token, ','); sys.omega2       = std::stod(token);
            std::getline(ss, token, ','); sys.Vsys         = std::stod(token);
            std::getline(ss, token, ','); sys.vexp         = std::stod(token);
            std::getline(ss, token, ','); sys.t            = std::stod(token);
            std::getline(ss, token, ','); sys.z            = std::stod(token);
            std::getline(ss, token, ','); sys.ffluid       = std::stod(token);
            std::getline(ss, token, ','); sys.M            = std::stod(token);
            std::getline(ss, token, ','); sys.r            = std::stod(token);
            std::getline(ss, token, ','); sys.B            = std::stod(token);
            std::getline(ss, token, ','); sys.Bcrit        = std::stod(token);
            std::getline(ss, token, ','); sys.rho_fluid    = std::stod(token);
            std::getline(ss, token, ','); sys.g_local      = std::stod(token);
            std::getline(ss, token, ','); sys.M_DM         = std::stod(token);
            std::getline(ss, token, ','); sys.delta_rho_rho = std::stod(token);
            systems.push_back(sys);
        }
    }
    return systems;
}
```

CSV Format (18 fields):

```
name,I,A,omega1,omega2,Vsys,vexp,t,z,ffluid,M,r,B,Bcrit,rho_fluid,g_local,M_DM,delta_rh
```

---

6. Command-Line I/O

```
int main(int argc, char** argv) {
    std::string input_file;
    std::string output_file;
    for (int i = 1; i < argc; i += 2) {
        std::string arg = argv[i];
        if (arg == "--input"  && i + 1 < argc) input_file  = argv[i + 1];
        if (arg == "--output" && i + 1 < argc) output_file = argv[i + 1];
    }
    // If --input given: load_muge_systems(input_file)
    // If --output given: redirect std::cout to file
}
```

---

7. Multi-File Architecture (Proposed)

Header decomposition for modular build:

```
main.cpp          — command-line entry, orchestration
```

```
celestial.h/cpp  – CelestialBody struct, Ug1/Ug2/Ug3/Ug4/Um/FU functions
muge.h/cpp       – MUGESystem, ResonanceParams, compressed+resonance MUGE
fluidsolver.h/cpp – FluidSolver (Navier-Stokes), simulate_quasar_jet
```

CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.10)
project(StarMagic)
set(CMAKE_CXX_STANDARD 11)
add_executable(star_magic main.cpp celestial.cpp muge.cpp fluidsolver.cpp)
```

---

8. Key Numerical Reference (Wormhole term across 7 systems)

System	r (m)	a_worm (m/s^2)
Magnetar SGR 1745-2900	1e4	7.09e-36 / 1e8 ≈ 7.09e-44
Sagittarius A*	1e12	7.09e-36 / 1e24 ≈ 7.09e-60
Tapestry of Blazing Starbirth	3.086e17	7.09e-36 / 9.52e34 ≈ 7.44e-71
Westerlund 2	3.086e17	same as Tapestry
Pillars of Creation	9.46e15	7.09e-36 / 8.95e31 ≈ 7.92e-68
Rings of Relativity	3.086e17	7.09e-36 / 9.52e34 ≈ 7.44e-71
Student's Guide	1e26	7.09e-36 / 1e52 ≈ 7.09e-88

The wormhole term is always subdominant (≪ other MUGE terms), confirming it acts as a geometrically-grounded perturbation rather than a dominant contribution.

---

9. CP4 Class

Class: `WormholeMUGETermImplSafetyCalculator` **Category:** Physics Implementation / Validation Infrastructure **References:** PAPER_373 (Morris-Thorne), PAPER_375 (a_worm formula suggestion)
